

Adding a Managed Component in ESP-IDF

Quick reference for reusing external components (e.g. sensor drivers) in any ESP-IDF project

1. Open the ESP-IDF Terminal

Don't use a plain PowerShell/CMD window — `idf.py` only exists on PATH inside a terminal where ESP-IDF's environment has been activated.

In VS Code: Command Palette → "ESP-IDF: Open ESP-IDF Terminal"

(Manual alternative: run `export.ps1` / `export.bat` from your IDF install folder in a normal terminal.)

2. Find the component and its version

Go to the official ESP-IDF Component Registry and search for the component you need. Each package page shows the latest version and the exact command to copy.

```
https://components.espressif.com
```

3. Run add-dependency for each component

From the ESP-IDF terminal, in your project root:

```
idf.py add-dependency "espressif/onewire_bus^1.0.4"  
idf.py add-dependency "espressif/ds18b20^0.1.2"
```

The ^ means "compatible with this version or newer, same major version" (semantic versioning). This writes/updates `main/idf_component.yml` automatically — no manual `CMakeLists.txt` edit needed for the dependency itself.

4. Confirm CMakeLists.txt registers your source + needed built-in components

`main/CMakeLists.txt` should look like:

```
idf_component_register(SRCS "main.c"  
                      INCLUDE_DIRS "."  
                      REQUIRES driver)
```

`REQUIRES driver` is only needed if you use built-in drivers like GPIO. External managed components (`onewire_bus`, `ds18b20`, etc.) don't need to be listed here — the component manager links them automatically based on `idf_component.yml`.

5. Build

```
idf.py build
```

This fetches the components, compiles everything, and generates `build/compile_commands.json` — which is what clears false "cannot open source file" squiggles in VS Code's IntelliSense.

Tip: repeat step 3 with a new package name any time you need another managed component — steps 1, 4 and 5 stay the same across projects.